ChessSR: Engine Architecture and Significance

Ishaan Maheshkumar

May 2026

1 Introduction

ChessSR is a chess engine that combines fast and efficient move generation and evaluation with Symbolic Regression, to detect flaws in the dataset, and use as the evaluation function itself.

2 Bit-boards and Bit-wise Operators

Bit-boards are used to represent almost all chess engines today, instead of using slow, fixed, arrays or double arrays to represent chess boards, 64 bit integers are used instead, 1 for every square on the board. This Bit-board, gives us a 0 or 1 value for every single piece on the board, we can use this to identify the occupancy of every piece on a board. Although this can not identify what piece is on a certain square, in order to do this, we create multiple 64 bit integers, one representing each color and piece, such as whitePawns, blackKnights, and we also create blackPieces, whitePieces, and allPieces. In order to change the state of the board, we use Bit-wise operators.

Bit-wise Operators (C++)

- Bit-wise and &
- Bit-wise or —
- Bit-wise not ˜
- Bit-wise exclusive or ˆ
- Bit-wise left shift << (shifts a number n bits to the left)
- Bit-wise right shift >> (shifts a number n bits to the right)

These operators are similar to boolean operators, they do the same exact thing, but for integers. Since each integer is just binary 0's and 1's, if we treat each 0 as false, and 1 as true, we can apply operators to the values of each in

each index in 2 given integers, and return one different integer. This bit-wise manipulation is extremely fast compared to other techniques.

Keep in mind, in C++, you must shift with 1ULL<<n, instead of 1<<n to preserve all the bits to the right of the ones, same thing for right shifting. using if(anyBitboard), returns true if the bit-board's value is>[]0 and false if it is 0. Also, use uint_64_t in C++ to create a 64 bit integer, that does not have a bit for negative numbers.

3 Move Generation

Move generation in chess utilizes Bit-wise Operators in order to make moves much faster than any other way out there. We use a series of operators to move pieces on the board.

3.1 Pawn Moves

Pawns can move in many ways, single push, double push, en-passant and captures, all for each color. To single push white pawns, we use

(whitePawns<<8)&˜allPieces

what this does, is it takes all the white pawns positions, moves them 8 to the left, which is 1 move forwards, and checks if it lands on an empty square (not on an occupied one). for black pawns, we use the same equation but we use right shift instead of left shift. (blackPawns>>8)&˜allPieces. This is because white pawns move up indices, while as black pawns move down, we do the same for double pushes, but 16 (2 spots forward) instead of 8. But we must also check if it is on the correct file. We do this by checking the spot ahead and the one ahead of that, and see if any are occupied, and we also check if the landing square lands on rank4 (a bit-board where rank 4 is all 1's), rank 4 is for white pawns. For en-passant we simply create a square index to track which square can be moved to via en-passant for that current position, and if possible, we allow it. and for captures, shifts of 7, and 9 are used to move diagonally and instead of checking if the square is empty, we check if the square lands on an opposing piece.

3.2 Knight and King moves

Knight and king moves are far simpler than pawn moves, simply take the position of the current piece, and an empty bit-board, and use the — operator to append 1's to the (1¡¡piece) shifted by 8 values, differing for kings and knights. The speed of this can be improved, by pre-computing the possible squares a piece can attack at any given square before hand, and during runtime, we simply check if the piece does not land on its own side's pieces, using the & method

used in pawns.

3.3 Knight, Rook, and, Queen Moves

There are 2 ways to generate these moves, the first of which is the easiest, we simply start at the staring square, and loop in all 4 directions(8 for queen) until each one hits a barrier, and we have all legal moves. But this is extremely slow, instead, we use a technique called magic bit-boards, this is a far more complicated hashing algorithm, we store all the possible attacks for every valid blocker position into a large table. Functions are created to find a magic number, to multiply the blocker position, and then shifted, to get the exact index of that position on that table. This way uses more ram to store the table, but is instant in calculating moves.

3.4 Legality Checking

To check if a move does not make the king stay in check, we simply make the move, and if it is illegal, we unmake it, we do this by finding all possible areas all pieces on a board can attack, and we combine them using —, and we see if the opposing king is in this bit-board, using & if so, it is in check and the move is illegal.

4 Search

Search is used to evaluate moves ahead, instead of 1 move deep. I used many algorithms to increase depth and get a better playing bot.

4.1 Basic Alpha Beta Search

Alpha Beta search is a recursive function, that calls itself until depth is 0, Alpha is a value representing the max value, the player (you) can make, Beta, is the minimizing player (opponent), or the opponent's best move (minimum value) when you find a branch worse then your current option, you prune it and do not search it any further.

4.2 Negamax function

Negamax is an addition to Alpha Beta/mini-max search algorithms meant for games like chess, Negamax says that the evaluation of a current board, is the same evaluation negated for the opponent. In practice, this means that the Negamax function evaluates each position from the perspective of the player to move, and the evaluation of the board is flipped (negated) when switching sides.

4.3 Move Ordering

Move ordering is one of the easiest optimizations for a chess engine's search algorithm, it uses the fact that alpha beta search and negate search take significantly less time to search through if the best moves are located in the front of the search, because the rest will get pruned. Because of this, we sort moves that are captures, and checks, in the front.

4.4 MVVLVA (Most valuable victim least valuable attacker)

MVVLVA is an addition to move ordering that makes it even more effective, we assign each capture move a value based on who is attacking, and who is getting captured, a queen capturing a pawn, is valued less then a pawn capturing a queen, this is because we do not want to put a high value piece in risk, and attacking a large piece with a small piece is almost guaranteed to be a good move. This heavily prunes moves, and in heavy depths, this saves exponential amounts of compute.

4.5 Quiescence Search

Quiescence Search is the processes of searching capture moves after all moves have been searched, this prevents the horizon effect, for example, The engine may be able to capture a knight with it's queen at its final depth, but it won't see that the queen could be captured the very next move, so the solution is, we keep checking every capture move on a board until there are none left, and choose the best path, so once normal move search is over, we run search on the set of capture moves, infinitely until search position is quiet.

4.6 Transposition tables

Transposition Tables are a small but helpful boost to engine speed, It simply stores every board trait, and its evaluation, so if the search encounters it again, it simply checks the table, for an instant evaluation instead of re evaluating.

4.7 Lazy SMP

Lazy SMP takes Transposition Tables to a whole new level by taking spare cores on the CPU, and telling it to begin searching on slightly different positions or depths, these cores server no purpose other than feeding positions right to the Transposition Table, so the main search core spends less time searching, and can get most of its moves right from the table in an instant.

4.8 Iterative deepening

Iterative deepening is a simple addition to search to increase speed by a decent amount, instead of searching right at depth(7), search is started at depth(1),

and depth (2) is calculated... and so on till depth(7), this seems way slower than normal search, although, many positions are recalculated, they put many positions into the Transposition Table, including ones from previous depths, making overall search a lot faster.

5 Symbolic regression integration

Symbolic regression Is used in this chess engine, for the evaluation, this is because I wanted to experiments it's use cases. A Symbolic formula is a a formula that uses many variables in the formula to produce a desired output. My engine uses 42 different functions to extract data from the board, and use that to form an equation so that every board can be evaluated to have the same output of stockfish's evaluation, in a condensed formula, although, there will always be some loss. The formula was still slower than Hand Crafted Evaluation, and still had not been able to play well as I had hoped, although, after editing the python script to use linear loss, instead of mean squared loss, the evaluation loss in centi-pawns, instantly halved. My next step is to sigmoid function the CSV data set of stock-fishes evaluations,in order for the engine to do even better.

6 Significance

There is a huge significance in using symbolic regression to improve neural networks. By extracting numerical values of patterns and using them to predict a neural network, we are seeing the significance of each value in the formula. Based on this significance, we can find out which part of the dataset is important, or even underused. For example, if I have a dog detection CNN, I am able to make functions that give me numerical values on how much of the dogs ears are like a dogs, and the same for nose, tail, eyes, and etc. Now if I made many formulas using those numbers using symbolic regression, and find out that ear values, and nose values, are heavily used in the formulas, we know that they serve importance, but if tail values do not show up at all, or very little, we know that tails are the CNN's weakness, and we can remove this, by adding more dog pictures of tails into the neural network. This also apply to LLMs and their emotional values etc. Using this, people can create true emotional LLMs and much more precises and consistent neural networks.